

SRM Institute of Science and Technology
College of Engineering and Technology
Department of Electronics and Communication Engineering
21IPE314P / RTOS Programming
Review – 3 Report (Final Evaluation)
Submission Date: 27.4.2026

Title of the project: **Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS**

Student 1 Reg no. and Name and Dept: **Naga Praneeth Reddy, RA2311004010473, ECE-H**

Student 2 Reg no. and Name and Dept: **Rudra Shukla, RA2311004010482, ECE-H**

A. Demo Evaluation (7.5 Marks)

S.No	Criteria	Description	Marks	Reg.No 473	Reg.No 482
1	Functionality	Complete system working	2		
2	Integration	Module integration	1		
3	Results	Accuracy of output	1.5		
4	Innovation	Key features	2		
5	Presentation	Demo explanation	1		

B. Viva Evaluation (5 Marks)

S.No	Criteria	Description	Marks	Reg.No 473	Reg.No 482
1	Concepts	Core understanding	2		
2	Technical	Tools/algorithms	1		
3	Contribution	Individual role	1		
4	Problem Solving	Application	1		

C. Report Evaluation (7.5 Marks)

S.No.	Criteria	Description	Marks	Reg.No 473	Reg.No 482
1	Structure	Organization	1		
2	Methodology	Implementation clarity	1.5		
3	Results	Analysis	2		
4	Diagrams	Figures/Flowcharts	1		
5	Conclusion	Future scope	1		
6	Quality	Grammar/ completeness	1		
Total Marks			20		

Course Teachers: Dr. S. Dhanalakshmi and Dr. Jaba Deva Krupa A

COLLEGE OF ENGINEERING & TECHNOLOGY
SRM INSTITUTE OF SCIENCE & TECHNOLOGY
S.R.M. Nagar, Kattankulathur – 603203, Chengalpattu District

BONAFIDE CERTIFICATE

Certified that this project report titled “**Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS**” is the bonafide work of “**Rudra Shukla(RA2311004010482), Naga Praneeth Reddy(RA2311004010472)**”, who carried out the project work under my supervision, as a part of the course “**21IPE314P / RTOS Programming**”, during the academic year 2025-26 (EVEN) in the Department of ECE, SRM Institute of Science & Technology, Kattankulathur.

SIGNATURE
Dr. S. Dhanalakshmi
Profesor
Dept. of Electronics and
Communication Engineering

SIGNATURE
Dr. Jaba Deva Krupa A
Assistant Professor
Dept. of Electronics and
Communication Engineering

DECLARATION

We hereby declare that the Project entitled “ **Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS** ” to be submitted for the course “**21IPE314P / RTOS Programming**”, is our original work as a team

Place:

Date:

[Student name] [Reg. No]

,[Student Name] [Reg. No]

Abstract

This report covers the design, implementation and hardware validation of a Hardware-in-the-Loop infusion pump controller prototype. The system runs QNX Neutrino RTOS 8.0.0 on a Raspberry Pi 4B as the real-time controller, with an STM32 Nucleo-F401RE microcontroller acting as the plant simulator. The two platforms communicate over a direct 3.3V UART link at 115200 baud using a fixed binary framing protocol with CRC8-MAXIM integrity checking. On the QNX side, three processes handle the system: `motor_ctrl` owns the UART and acts as the control server, `demo_client` sends commands using `MsgSend` and `MsgReply`, and `alarm_mgr` receives fault notifications through `MsgSendPulse`. Development was split into four incremental stages, each validated on hardware before the next stage began. The four stages cover the protocol layer, the message-passing server, asynchronous fault notification, and continuous heartbeat monitoring. All four stages passed hardware validation on 2026-04-14. The project is scoped as a safety-informed classroom prototype and draws on design principles from mixed-criticality embedded systems, implemented on a real RTOS with real hardware interaction.

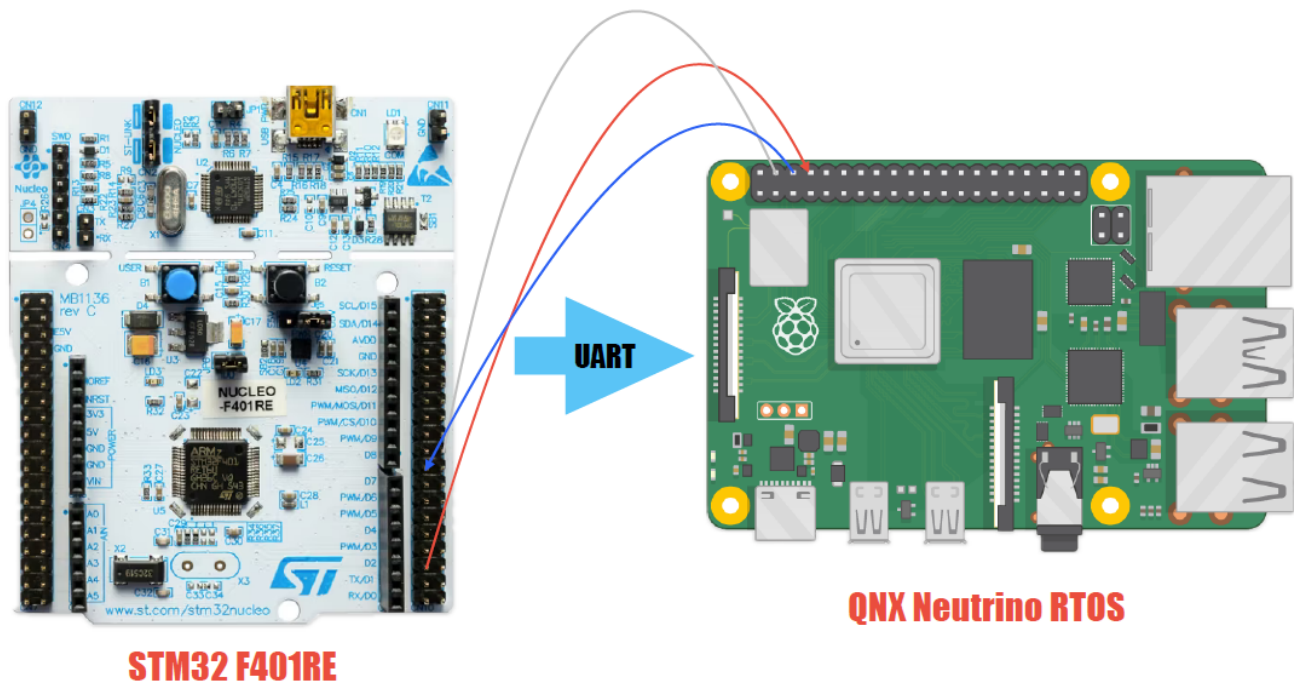
TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	
	ABSTRACT	
1	Project Overview	
2	System Architecture	
3	Protocol Design and Methodology	
4	Module Development	
5	Tools and Technology	
6	Results and Testing	
7	Problem Solving and Debugging	
8	Conclusion and Future Work	20
	REFERENCES	21

1. Project Overview

This project implements a Hardware-in-the-Loop infusion pump controller prototype using QNX Neutrino RTOS on a Raspberry Pi 4B as the real-time controller, with an STM32 Nucleo-F401RE microcontroller acting as the hardware-side plant simulator. The two platforms communicate over a direct 3.3V UART link at 115200 baud, and all inter-process coordination within QNX is handled through the operating system's native message-passing mechanisms.

The motivation for choosing QNX as the target platform comes from its microkernel architecture and its suitability for mixed-criticality embedded applications. Rather than attempting to build a fully IEC 62304-certified implementation, this project is deliberately scoped as a safety-informed classroom prototype. The design priorities are a reliable hardware-validated communication path, a cleanly separated QNX process architecture, and demonstrable fault detection behaviour. These goals have been fully achieved through a sequence of incrementally validated implementation stages.

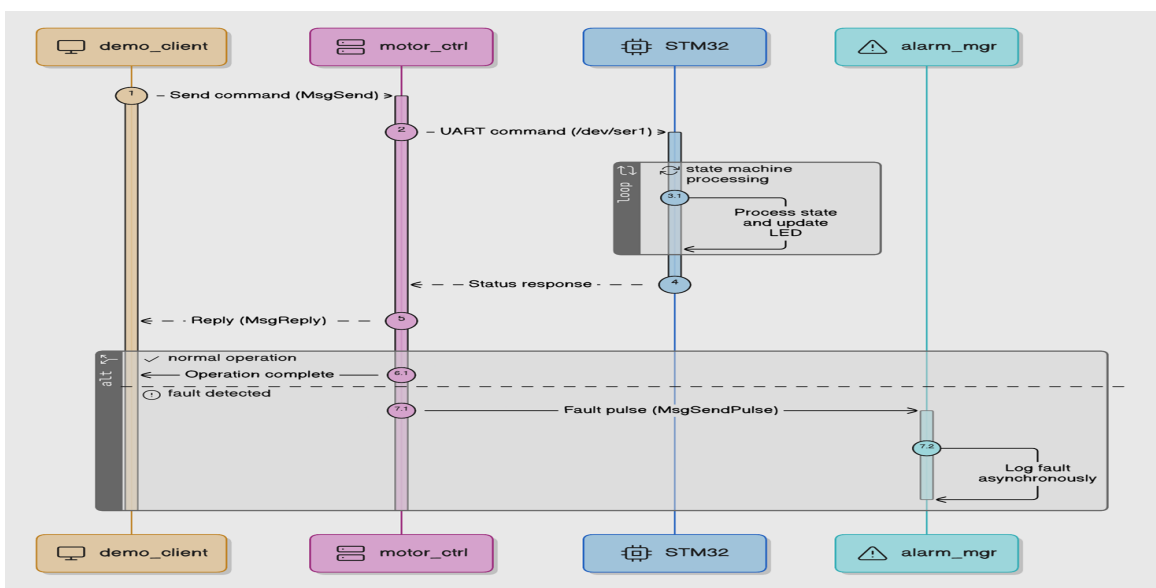


2. System Architecture

The system is structured around three QNX processes and one STM32 firmware component. On the QNX side, `motor_ctrl` acts as the central control server. It exclusively owns the `/dev/ser1` serial device and is the only process that communicates directly with the STM32. A small client application called `demo_client` sends `START`, `STOP`, and `STATUS` commands to `motor_ctrl` using `MsgSend`, and receives synchronous replies via `MsgReply`. A separate `alarm_mgr` process runs independently, waiting on its own channel for fault pulses that `motor_ctrl` delivers asynchronously using `MsgSendPulse`.

This design deliberately avoids implementing a custom QNX resource manager. A resource manager would have required significantly more development time for POSIX-interface boilerplate that adds no visible demo value. The message-passing server approach chosen here achieves the same goal of centralised UART ownership with a much simpler implementation, while still exercising the core QNX IPC primitives. The separation between synchronous command control and asynchronous fault notification is a key architectural feature: `demo_client` is never blocked waiting for an alarm, and `alarm_mgr` receives fault notifications independently of whether any client command is in flight.

The STM32 firmware runs a simple four-state machine covering `IDLE`, `RUNNING`, `FAULT`, and `EMERGENCY_STOP` transitions. The onboard LED provides continuous visual feedback of the current state: off for `IDLE`, solid on for `RUNNING`, and blinking for `FAULT`. The user button on the Nucleo board serves as a manual fault injection source for demonstration purposes.

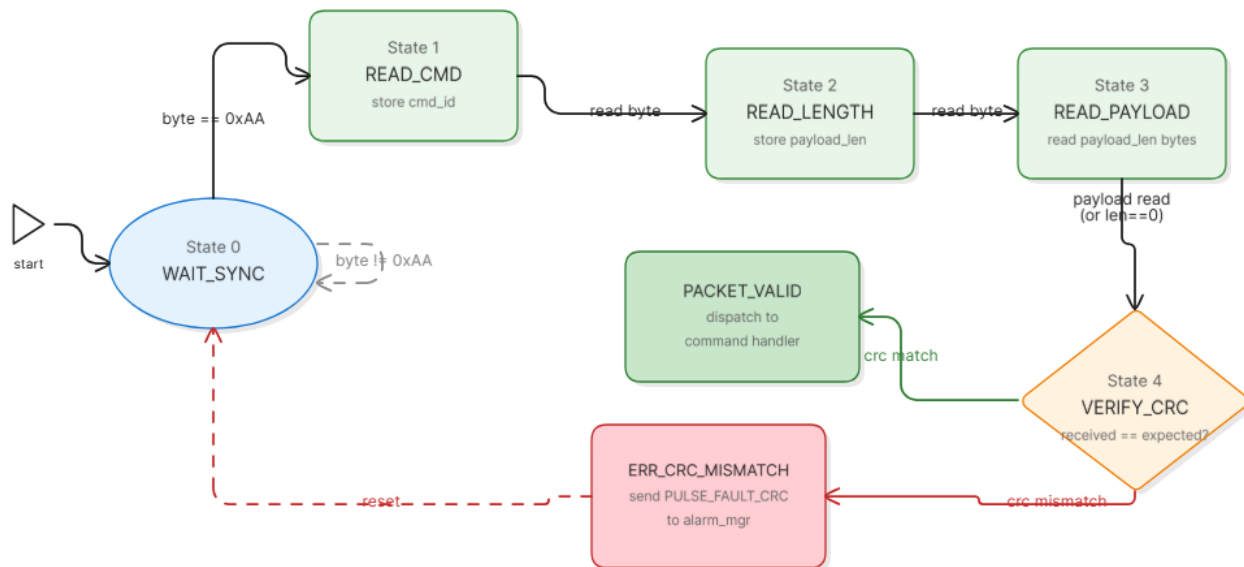


3. Protocol Design and Methodology

All UART communication between QNX and the STM32 uses a fixed binary framing structure. Each packet consists of a sync byte (0xAA), a one-byte command ID, a one-byte payload length, an optional payload of up to 16 bytes, and a trailing CRC8 checksum. The CRC is computed over the command, length and payload bytes using the CRC8-MAXIM (Dallas) algorithm in its LSB-first reflected form with polynomial 0x8C. The same CRC function is compiled into both the QNX and STM32 codebases, so any byte corruption or framing error is caught at the receiver before a command is acted on.

Packet parsing is handled by a byte-level state machine rather than a single blocking read call. This is necessary because the QNX serial driver does not guarantee that a read() call returns all requested bytes in one shot. The parser moves through five states sequentially:

Packet Receiver State Machine

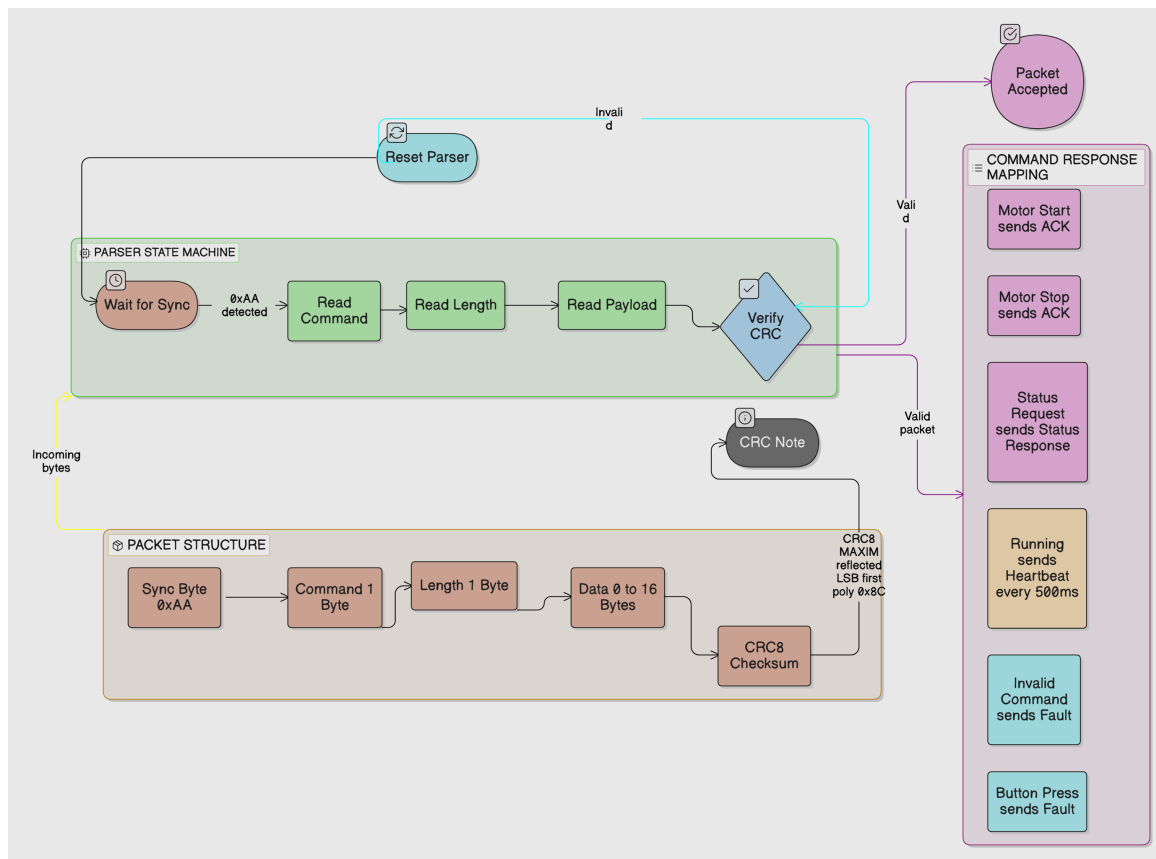


A CRC mismatch at State 4 resets the parser to State 0 and sends a fault pulse to alarm_mgr via MsgSendPulse. On the write side, a write_full() helper loops over write() until all bytes of the packet are confirmed transmitted, which was added after partial-write issues were observed on the QNX serial driver during bring-up (see Section 7).

The command set used we defined consists :-

- MOTOR_START (0x01)
- MOTOR_STOP (0x02)
- STATUS_REQ (0x03)
- HEARTBEAT (0x04)
- ACK (0x05)
- FAULT (0x06)
- STATUS_RSP (0x07).

The STM32 responds to START and STOP with an ACK packet, answers STATUS_REQ with a STATUS_RSP packet carrying a one-byte motor state value, sends HEARTBEAT packets at 500ms intervals while in RUNNING state, and sends a FAULT packet either in response to an invalid command or when the user button is pressed.



4. Module Development

Development was organised into discrete implementation stages, each producing a fully tested and hardware-validated snapshot before the next stage began. This approach kept debugging tractable and ensured that a working fallback always existed.

The first functional stage established the framed protocol layer. A standalone QNX test program sent START, STOP, and STATUS commands as framed packets and validated the STM32 responses against expected reply types. This stage confirmed that CRC calculation matched across both platforms and that the parser handled all three request/reply flows correctly under real hardware conditions.

The second stage introduced the message-passing motor_ctrl server. UART ownership was moved inside motor_ctrl, and a demo_client was built to interact with it through MsgSend and MsgReply. This stage validated the complete synchronous control path: demo_client sends a typed request structure, motor_ctrl translates it into a UART packet, waits for the STM32 response, and returns a structured reply to the client. The client never accesses /dev/ser1 directly.

The third stage added alarm_mgr and introduced asynchronous fault notification. When motor_ctrl detects a fault condition, it sends a pulse to alarm_mgr using MsgSendPulse. Three fault conditions are handled: UART timeout, CRC or protocol mismatch, and an explicit FAULT packet from the STM32. alarm_mgr receives these pulses independently of any pending client request and logs the fault code and value to the terminal.

The fourth and most complex stage added continuous heartbeat monitoring. motor_ctrl now runs two background threads in addition to its main MsgReceive dispatch loop. A dedicated UART RX thread reads incoming bytes continuously and feeds them to the packet parser, updating a shared heartbeat timestamp each time a valid HEARTBEAT packet arrives. A separate heartbeat monitor thread wakes every 200ms, checks the elapsed time since the last heartbeat, and sends a PULSE_FAULT_HEARTBEAT_MISS to alarm_mgr if no heartbeat has been seen for more than two seconds. Client requests issued after a heartbeat miss are rejected with a HEARTBEAT_MISS status code rather than attempting a UART transaction against a disconnected or faulted device.

5. Tools and Technology

Component	Specification / Version	Role
Raspberry Pi 4B	4GB RAM, AArch64, QNX Neutrino 8.0.0	QNX RTOS controller
STM32 Nucleo-F401RE	ARM Cortex-M4, 84MHz, USART1	HIL plant simulator
QNX SDP 8.0	Momentics IDE, GCC cross-compiler	Build and deploy toolchain
STM32CubeIDE 1.14+	HAL, USART1 interrupt driver	STM32 firmware development
UART Link	115200 baud, 3.3V, PA9/PA10	Physical communication layer
QNX IPC APIs	name_attach, MsgSend, MsgSendPulse	Inter-process communication
CRC8-MAXIM	Polynomial 0x8C, reflected LSB-first	Packet integrity verification

Cross-compilation for the QNX AArch64 target was handled entirely within QNX Momentics IDE. Binaries were deployed to the Raspberry Pi using SCP over SSH, with the Pi's network interface configured manually after each boot using a fixed IP address of 192.168.1.50. The QNX filesystem is read-only at the root level, so all runtime binaries and writable state were placed in the /var RAM filesystem.

6. Results and Testing

All four implementation stages were hardware-validated on the actual QNX and STM32 setup on 2026-04-14. The table below lists each test case, the expected behaviour based on the protocol and architecture design, the observed output from terminal logs, and the result.

#	Test Case	Expected Behaviour	Observed Output	Result
1	START command, protocol layer	TX: AA 01 00 C4, RX: AA 05 00 FF (ACK)	TX AA 01 00 C4 / RX AA 05 00 FF [VALID] / Response CMD: 0x05	PASS
2	STATUS_REQ, protocol layer	RX: AA 07 01 01 E0, data byte = RUNNING (0x01)	AA 07 01 01 E0 received, data byte decoded as RUNNING	PASS
3	STOP command, protocol layer	TX: AA 02 00 91, RX: ACK, LED turns off	Clean ACK received, STM32 LED turned off	PASS
4	CRC consistency	No CRC mismatch on valid packet exchanges	No CRC mismatch observed across all valid exchanges	PASS
5	demo_client START, message-passing layer	Reply status OK (0), Response cmd 0x05	Reply status: OK (0) / Response cmd: 0x05 / Data len: 0	PASS
6	demo_client STATUS, message-passing layer	Reply status OK (0), Response cmd 0x07, motor state RUNNING	Motor state: RUNNING decoded from STATUS_RSP correctly	PASS
7	demo_client STOP, message-passing layer	Reply status OK (0), Response cmd 0x05	Reply status: OK (0) / Response cmd: 0x05	PASS
8	Fault notification, user button press	demo_client returns REMOTE_FAULT (-4), alarm_mgr independently logs the pulse	REMOTE_FAULT (-4) on client / alarm_mgr: REMOTE_FAULT (code=3 value=1)	PASS
9	Fault pulse independence	alarm_mgr pulse arrives regardless of client request state	Pulse confirmed delivered to alarm_mgr at same time as client reply	PASS
10	Heartbeat miss detection	alarm_mgr logs HEARTBEAT_MISS within 2000ms of UART interruption	alarm_mgr: HEARTBEAT_MISS (code=4 value=2000) within approximately 2 seconds	PASS
11	Client rejection after heartbeat miss	All subsequent client requests rejected with status -6	UNKNOWN (-6) returned for start, status and stop after miss	PASS

Protocol layer results

The CRC8-MAXIM implementation produced consistent checksums across both the QNX AArch64 and STM32 Cortex-M4 builds without any adjustment. The START packet checksum (AA 01 00 C4) and the STATUS_REQ checksum (AA 03 00 55) were both verified against manual calculations and matched on the first hardware test. The STOP command (AA 02 00 91) returned a clean ACK and the STM32 LED turned off, confirming that the state machine on the STM32 side transitioned correctly.

```
Command Prompt - ssh -i C:\  ×  Windows PowerShell  ×  +  v
409623    2 usr/sbin/qconn          10r RECEIVE          1
417816    1 usr/sbin/qconn          10r SIGWAITINFO
417816    2 usr/sbin/qconn          10r RECEIVE          1
# chmod +x /var/slice2_test
# ls -la /var/slice2_test
-rwxr-xr-x  1 root root 25488 1970-01-01 01:10 /var/slice2_test
# slay -f cat
# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4

# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4
RX: AA 05 00 FF [VALID]
Response CMD: 0x05 (ACK)

*** SLICE 2 TEST PASSED ***
# /var/slice2_test start
=== Slice 2 Protocol Test ===
Port: /dev/ser1
Command: start (0x01)

TX [4 bytes]: AA 01 00 C4
RX: AA 05 00 FF [VALID]
Response CMD: 0x05 (ACK)

*** SLICE 2 TEST PASSED ***
```

Message-passing layer results

The full command path from demo_client through motor_ctrl over UART to the STM32 and back completed correctly for all three command types. The response codes 0x05 for ACK and 0x07 for STATUS_RSP confirm that the dedicated STATUS_RSP command introduced during debugging successfully resolved the response ambiguity that existed when all replies were being sent as ACK packets. The client never accessed /dev/ser1 directly in any test, confirming that UART ownership remained exclusively inside motor_ctrl throughout.

CLIENT:-

```
Command Prompt - ssh -i C:\l  X  Command Prompt - ssh -i C:\  X  Windows PowerShell  X  +

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\shukl>ssh -i %USERPROFILE%\ssh\id_ed25519 root@192.168.1.50
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: OK (0)
Response cmd: 0x07
Data len: 1
Motor state: RUNNING
# /var/demo_client stop
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: OK (0)
Response cmd: 0x07
Data len: 1
Motor state: RUNNING
# /var/demo_client stop
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# |
```

SERVER (MOTOR CONTROL):-

```
Command Prompt - ssh -i C:\  x  Command Prompt - ssh -i C:\  x  Windows PowerShell  x  +  v
# chmod +x /var/demo_client
# ls -la /var/motor_ctrl /var/demo_client
-rwxr-xr-x 1 root root 17464 1970-01-01 02:49 /var/demo_client
-rwxr-xr-x 1 root root 27536 1970-01-01 02:49 /var/motor_ctrl
# slay -f cat
# /var/motor_ctrl
motor_ctrl ready on name 'pump_motor_ctrl'
UART port: /dev/ser1
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 03 00 55
RX byte: AA
RX byte: 07
RX byte: 01
RX byte: 01
RX byte: E0
TX [4 bytes]: AA 02 00 91
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 03 00 55
RX byte: AA
RX byte: 07
RX byte: 01
RX byte: 01
RX byte: E0
TX [4 bytes]: AA 02 00 91
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
```

Fault notification results

Test cases 8 and 9 together confirm the key property of the three-process architecture: the fault pulse sent via MsgSendPulse reached alarm_mgr at the same time as the client received its REMOTE_FAULT (-4) reply. The client reply path and the alarm notification path operated independently, which is the main safety-relevant behaviour this architecture is designed to achieve.

ALARM MANAGER:-

```
# slay -f motor_ctrl
# slay -f demo_client
# slay -f alarm_mgr
# rm /var/motor_ctrl
# rm /var/demo_client
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x 1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rw-r--r-- 1 root root 19432 1970-01-01 03:56 /var/demo_client
-rw-r--r-- 1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# chmod +x /var/motor_ctrl /var/demo_client
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x 1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rwxr-xr-x 1 root root 19432 1970-01-01 03:56 /var/demo_client
-rwxr-xr-x 1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/alarm_mgr
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)
```

SERVER (MOTOR CONTROL):-

```
# /var/motor_ctrl
motor_ctrl ready on name 'pump_motor_ctrl'
UART port: /dev/ser1
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 05
RX byte: 00
RX byte: FF
TX [4 bytes]: AA 01 00 C4
RX byte: AA
RX byte: 06
RX byte: 00
RX byte: AA
STM32 reported FAULT packet
alarm pulse sent: code=3 value=1
```

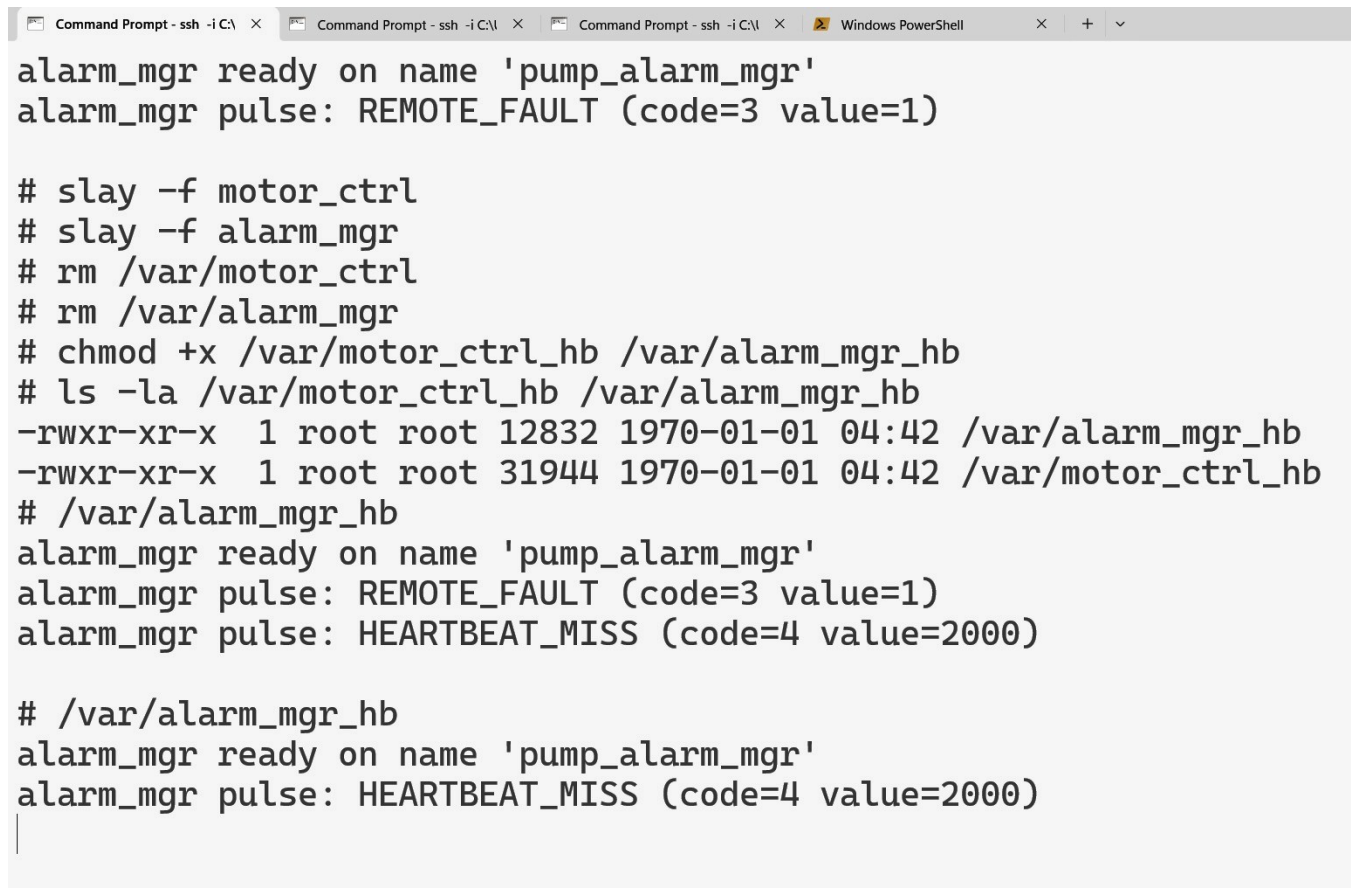
CLIENT:-

```
# ls -la /var/alarm_mgr /var/motor_ctrl /var/demo_client
-rwxr-xr-x  1 root root 12832 1970-01-01 03:50 /var/alarm_mgr
-rwxr-xr-x  1 root root 19432 1970-01-01 03:56 /var/demo_client
-rwxr-xr-x  1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: REMOTE_FAULT (-4)
Response cmd: 0x00
Data len: 0
# |
```

Heartbeat monitoring results

The heartbeat monitor thread is configured to poll every 200ms and declare a miss after 2000ms without a valid HEARTBEAT packet. The observed pulse value of 2000 in the HEARTBEAT_MISS notification confirms that the elapsed time measurement was working correctly. After the miss was detected, all three subsequent client operations (start, status, stop) returned UNKNOWN (-6), confirming that the heartbeat_missed flag was checked consistently across all command handlers in motor_ctrl.

ALARM MANAGER:-

A screenshot of a terminal window with multiple tabs. The active tab is titled 'Command Prompt - ssh -i C:\'. The terminal displays the following commands and their outputs:

```
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)

# slay -f motor_ctrl
# slay -f alarm_mgr
# rm /var/motor_ctrl
# rm /var/alarm_mgr
# chmod +x /var/motor_ctrl_hb /var/alarm_mgr_hb
# ls -la /var/motor_ctrl_hb /var/alarm_mgr_hb
-rwxr-xr-x  1 root root 12832 1970-01-01 04:42 /var/alarm_mgr_hb
-rwxr-xr-x  1 root root 31944 1970-01-01 04:42 /var/motor_ctrl_hb
# /var/alarm_mgr_hb
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: REMOTE_FAULT (code=3 value=1)
alarm_mgr pulse: HEARTBEAT_MISS (code=4 value=2000)

# /var/hb
alarm_mgr ready on name 'pump_alarm_mgr'
alarm_mgr pulse: HEARTBEAT_MISS (code=4 value=2000)
```

SERVER (MOTOR CONTROL):-

[illegible]

CLIENT:-

```
-rwxr-xr-x 1 root root 28144 1970-01-01 03:56 /var/motor_ctrl
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: REMOTE_FAULT (-4)
Response cmd: 0x00
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client start
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
# /var/demo_client start
Reply status: OK (0)
Response cmd: 0x05
Data len: 0
# /var/demo_client status
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
# /var/demo_client stop
Reply status: UNKNOWN (-6)
Response cmd: 0x00
Data len: 0
```

7. Problem Solving and Debugging

Several issues were encountered and resolved during hardware bring-up that are worth documenting both for their debugging value and as evidence of the iterative engineering process.

During initial protocol testing, the STM32 responded to the first START packet by entering FAULT state and blinking the LED rather than returning an ACK. The UART transmission was confirmed working because the STM32 clearly reacted. Investigation of the STM32 state machine revealed that when motor_state was already FAULT or RUNNING from a previous session, the handle_packet function for CMD_MOTOR_START returned a FAULT packet rather than proceeding to the ACK path. The fix was to reset the STM32 to IDLE state before each test, and a permanent fix was to add an explicit state check at the start of the START handler. The STATUS_RSP response was also originally ambiguous because an early design overloaded the ACK command for all responses. A dedicated CMD_STATUS_RSP command was introduced to make response matching unambiguous.

The QNX write path originally assumed that a single write() call would transmit the complete packet. On the QNX serial driver this is not guaranteed. A write_full helper function was introduced that loops until all bytes are confirmed written, which eliminated intermittent partial-packet transmission issues.

STM32 UART packet handling was originally executed directly inside the HAL_UART_RxCpltCallback interrupt handler. This caused timing issues when the callback attempted to call HAL_UART_Transmit while an ongoing receive was still in progress on the same peripheral. Moving all command execution into the main loop and keeping the interrupt handler strictly as a byte feeder into the parser resolved this interference.

During deployment, a naming inconsistency between the Momentics binary output names and the expected filenames on the Pi caused confusion where old binaries remained in place after SCP appeared to succeed. The root cause was that running processes held file locks. Explicitly running slay followed by rm before each SCP transfer resolved this reliably.

The heartbeat monitor thread required careful mutex discipline because the UART RX thread and the heartbeat monitor thread both access the last_heartbeat_ms timestamp. A shared pthread_mutex protects all reads and writes to the controller state structure. The condition variable used in the main dispatch loop allows the heartbeat monitor to signal the dispatch thread when a response arrives, avoiding busy-wait polling during UART transactions.

8. Conclusion and Future Work

This project has delivered a hardware-validated QNX-based infusion pump controller prototype across four incremental stages, all tested on physical hardware. The protocol layer, the message-passing control server, the asynchronous alarm notification system and the heartbeat monitoring mechanism have each been validated individually and shown to work correctly as an integrated system.

From the perspective of the RTOS Programming course, the project demonstrates how QNX Neutrino's IPC mechanisms can be used to separate concerns across processes: synchronous command control through `MsgSend` and `MsgReply`, asynchronous fault propagation through `MsgSendPulse`, and hardware ownership isolation by restricting `/dev/ser1` access to `motor_ctrl`. These were not tested in simulation; they ran on a Raspberry Pi 4B talking to a real STM32 over a physical UART link.

The decision to use a message-passing server rather than a full QNX resource manager reduced implementation complexity without giving up the QNX-native IPC story. The incremental stage approach meant a working system existed at every point in development, which made debugging manageable and kept a fallback available at all times.

Several limitations exist in the current implementation. The system handles one client request at a time with no queuing mechanism. This is acceptable for a demo but would need to be addressed for any multi-client scenario. Once the `heartbeat_missed` flag is set, client requests fail indefinitely until `motor_ctrl` is restarted; a production system would need a recovery path involving a `MOTOR_STOP` command followed by a reconnection handshake. The QNX Adaptive Partitioning Scheduler was not configured in this phase, so CPU budget enforcement across `motor_ctrl`, `alarm_mgr` and any stress process has not been tested. The QNX secpol security model and the QNX Hypervisor virtualisation layer were explicitly deferred given the prototype scope and the project timeline.

The planned extensions build directly on what was validated here. APS partition configuration would give `motor_ctrl` a guaranteed CPU budget under system load, which matters in the infusion pump context. Tracelogger-based interrupt latency measurement would produce actual timing numbers rather than approximate observations, which is the kind of evidence IEC 62304 would require for a real medical device. A multi-client architecture with request queuing would be needed if multiple monitoring processes need concurrent access to pump status. Implementing the secpol model would enforce UART ownership at the OS level rather than relying on convention between processes.

References

- [1] QNX Software Systems, QNX Neutrino RTOS System Architecture, QNX Software Development Platform 8.0, Blackberry Limited, 2023.
- [2] QNX Software Systems, QNX Neutrino RTOS Programmer's Guide: Message Passing, QNX SDP 8.0, Blackberry Limited, 2023.
- [3] STMicroelectronics, UM1724 User Manual: STM32 Nucleo-64 Boards (MB1136), Rev 14, STMicroelectronics, 2023.
- [4] STMicroelectronics, STM32F401xB/STM32F401xC Reference Manual, RM0368, STMicroelectronics, 2023.
- [5] Dallas Semiconductor, Application Note 27: Understanding and Using Cyclic Redundancy Checks with Dallas Semiconductor iButton Products, Maxim Integrated, 2001.
- [6] IEC, IEC 62304:2006+AMD1:2015 Medical Device Software: Software Life Cycle Processes, International Electrotechnical Commission, 2015.